

ISM2411 — Lab Week 15

Aggregate & Chart — Capstone Warm-up — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 15 · Unit 4 · Data Analysis with pandas

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Segments	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Exercise 1 — Q1: Revenue by Region (0:04–0:14, 10 min)	4
6.3 Exercise 2 — Q2: Monthly Revenue Trend (0:14–0:24, 10 min)	7
6.4 Exercise 3 — Q3: Order Size Distribution (0:24–0:33, 9 min)	10
6.5 Exercise 4 — Q4: Your Own Business Question (0:33–0:41, 8 min)	12
6.6 Exercise 5 — Seaborn Version (0:41–0:48, 7 min)	14
6.7 Exercise 6 — Multiple Aggregations (0:48–0:55, 7 min)	16
6.8 Exercise 7 — Notebook Tidy-Up (0:55–0:59, 4 min)	17
7 Wrap-Up (last ~6 minutes)	19
8 Appendix A — Full Answer Key (aggregate.py)	19
9 Appendix B — Reproducible Full-Year Dataset (for instructor testing)	22
10 Appendix C — Extra Practice (only if the class finishes early)	23

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 15 Lab — Aggregate & Chart: Capstone Warm-up
Unit	Unit 4 · Data Analysis with pandas
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 14's <code>clean_sales.csv</code> ; Module 13's groupby-adjacent filtering and aggregation basics
Student-facing lab page	markumreed.github.io/ism2411 — week15_lab
Exercises covered	Exercises 1–7 (required)
Submission	<code>aggregate.ipynb</code> (or <code>.py</code>) + all 4 PNG chart files via GitHub (<code>module15/</code> folder), URL to Canvas

The lab page's own framing is worth stating to the class verbatim and taking seriously: “this is the capstone dress rehearsal — the workflow, standards, and deliverable format are identical.” Everything graded loosely in earlier modules — vague comments, unlabeled output — is graded strictly here, on purpose, because the capstone won't give partial credit for a chart with no title. Two things deserve outsized attention: **the “state your finding as a sentence with a number” discipline** (every required chart needs a one-sentence, numeric finding beneath it) and **Exercise 2's “all 12 months” requirement**, which contains a genuine, easy-to-miss gotcha — groupby silently omits any month with zero orders unless explicitly told not to.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Aggregate a DataFrame with `.groupby(...)[...].sum()` (or `.mean()`), and chain `.reset_index()` to get a clean, chart-ready result.
2. Build a labeled, titled bar chart, line chart, and histogram with matplotlib, and know which chart type fits which kind of question.
3. Ensure a time-based aggregation shows every expected period (e.g., all 12 months), even when some have zero activity.
4. State an analytical finding as a specific, numeric sentence — “Region X drove \$Y, Z% above the average” — not a vague description.
5. Use `.agg()` to compute multiple statistics from one groupby call.

6. Build an equivalent chart in Seaborn and articulate at least one concrete difference from the matplotlib version.

3 Before Class — Setup Checklist

- ☐ Confirm `matplotlib` and `seaborn` are installed on your demo machine, and rehearse Exercises 1–3 once end to end before class — chart-heavy live coding has more moving parts (figure setup, saving, closing) than most prior labs, worth the extra rehearsal.
- ☐ Obtain or generate `data/clean_sales.csv` — the direct output of Module 14. Appendix B documents a verified, reproducible synthetic dataset (1,200 rows, a full year, a genuine December seasonal peak, and a region — East — that wins on revenue and average order value despite *not* having the most orders, specifically so Exercise 6's two questions have two different correct answers) for your own pre-class testing. **Use your course's real Module 14 output with students.**
- ☐ Create a `module15/charts/` folder structure before class, and decide whether your section works in a `.py` file or a Jupyter notebook — Exercise 7's tidy-up instructions differ slightly by format, so settle this up front.

4 Materials Needed

- Instructor laptop + terminal + editor (or Jupyter), Python 3.10+, `pandas`, `matplotlib`, `seaborn` installed
- Students: `data/clean_sales.csv` from Module 14, same GitHub repo with a new `module15/` folder

5 Segments

The required exercises fill most of the available time.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Everything today is graded to the same standard as the capstone — because this lab is the capstone, at smaller scale, as a rehearsal. Every chart needs a title, axis labels, and a one-sentence finding stated as a real number underneath it. Not ‘revenue varies by region’ — ‘the South drove \$112,040 in revenue, the most of any region.’ That specificity is the actual skill this whole unit has been building toward.”

Do: Open `aggregate.py` (or a new notebook), type the header:

```
# aggregate.py
# ISM2411 Module 15 Lab – Aggregate & Chart: Capstone Warm-up
import pandas as pd
import matplotlib.pyplot as plt
```

6.2 Exercise 1 — Q1: Revenue by Region (0:04–0:14, 10 min)

Teaching goal: `.groupby().sum().reset_index()` as a repeatable three-step recipe, and a first properly-labeled bar chart — the template every remaining chart in this lab (and the capstone) will follow.

Say to the class:

“First business question: which region makes the most money? One aggregation, one bar chart, one sentence stating the answer as a number.”

Live-code this:

```
# --- Exercise 1 (Q1) ---
df = pd.read_csv('data/clean_sales.csv')

region_revenue = df.groupby("region")["revenue"].sum().reset_index()
region_revenue = region_revenue.sort_values("revenue", ascending=False)
print(region_revenue)

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(region_revenue["region"], region_revenue["revenue"], color="#4C72B0")
ax.set_title("Total Revenue by Region")
ax.set_xlabel("Region")
```

```
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q1_revenue_by_region.png", dpi=150)
plt.show()
```

Line-by-line explanation:

- `df.groupby("region")["revenue"].sum()` — read this as three separate steps, left to right: **group** the DataFrame into buckets, one per distinct region value (Module 13's `.unique()` values become the buckets); pull out just the "revenue" column from each bucket; **sum** it within each bucket. The result is one total per region.
- `.reset_index()` — **this is worth explaining rather than treating as boilerplate**. A raw `groupby(...).sum()` result uses the grouped column (region) *as its index*, not as an ordinary column — which works fine for some purposes but is awkward for charting and sorting. `.reset_index()` converts that index back into a normal column, producing a clean, two-column DataFrame (region, revenue) — say explicitly: **this specific “groupby, then reset_index” combination is worth memorizing as a unit**, since it's the standard shape almost every aggregation-for-charting task in this lab (and the capstone) will need.
- `.sort_values("revenue", ascending=False)` — Module 13's familiar sort, now applied to the aggregated result, so both the printed table and the resulting chart show regions in a meaningful order (highest revenue first) rather than whatever arbitrary order groupby happened to produce.
- `fig, ax = plt.subplots(figsize=(8, 5))` — **new syntax today**. This creates a matplotlib **figure** (fig, the overall canvas) and an **axes** (ax, the actual plotting area within it) together, in one call — say explicitly: nearly everything from here forward is built by calling methods *on* ax, not on plt directly; this fig, ax pattern is the standard, professional way to build a matplotlib chart, and it's worth using consistently rather than the simpler-looking but less flexible `plt.bar(...)` shortcut some tutorials show.
- `ax.bar(region_revenue["region"], region_revenue["revenue"], color="#4C72B0")` — the actual chart: region values become the x-axis categories, revenue values become the bar heights.
- `ax.set_title(...), ax.set_xlabel(...), ax.set_ylabel(...)` — **all three are required by this exercise, explicitly** — say plainly: an unlabeled chart, even a technically correct one, does not meet this lab's (or the capstone's) standard.
- `plt.tight_layout()` — automatically adjusts spacing so labels/titles don't get visually cut off or overlap — a cheap, worthwhile habit to include on every chart.
- `plt.savefig("module15/charts/q1_revenue_by_region.png", dpi=150)` — **must come before** `plt.show()` (or replace it entirely in a non-interactive script) — say explicitly why: `plt.show()` (or closing an interactive window) can clear the current figure from memory in some environments, so saving first guarantees the file is written correctly regardless.

Run it. Expected output (verified against the synthetic dataset in Appendix B — actual values will differ with real course data):

	region	revenue
0	East	158621.95
2	South	112040.69
3	West	63666.44
1	North	60112.83

Required one-sentence finding, written beneath the chart code (model this explicitly, since it's graded): "The East region drove \$158,621.95 in revenue, the highest of any region — about 41% more than the second-place South region."

Common student mistakes to watch for:

- Skipping `.reset_index()` and then being confused why `region_revenue["region"]` raises `KeyError` — without it, `region` is the DataFrame's *index*, not a column reachable by that bracket syntax; a good, concrete illustration of why the step matters, not just an instruction to follow.
- Forgetting one of the three required labels (title, x-label, y-label) — walk the room and check for all three explicitly, since it's an easy thing to skip under time pressure and is graded strictly here on purpose.
- Vague findings ("revenue differs by region") instead of the required specific, numeric sentence — redirect to the model sentence above.

Check for understanding: "If two regions had exactly equal revenue, what would this chart's sort produce?" (Their relative order would follow whatever tie-breaking `.sort_values()` applies by default — not something to derive precisely, but a good prompt that "sorted" doesn't always imply a unique, unambiguous order when values can tie.)

6.3 Exercise 2 — Q2: Monthly Revenue Trend (0:14–0:24, 10 min)

Teaching goal: Extract a month from a datetime column, aggregate by it, and confront a genuine, easy-to-miss gotcha: `groupby` only shows months that actually appear in the data — a month with zero orders simply vanishes from the result unless explicitly handled.

Say to the class:

“Second question: how does revenue move across the year? The lab page requires all 12 months on the x-axis — and I want to show you why that requirement isn’t automatic, because it genuinely trips people up.”

Live-code this:

```
# --- Exercise 2 (Q2) ---
df['order_date'] = pd.to_datetime(df['order_date'])
df['month'] = df['order_date'].dt.month

monthly = df.groupby("month")["revenue"].sum()
print(monthly)
```

Line-by-line explanation:

- `pd.to_datetime(df['order_date'])` — parses the date column to a real datetime type — say explicitly, echoing Module 14 Exercise 7’s gotcha: if `clean_sales.csv` was saved and reloaded without `parse_dates`, `order_date` likely came back as object (text), not `datetime64` — this line (re-)converts it, whether or not that specific gotcha applies to your students’ actual file.
- `df['order_date'].dt.month` — `.dt` is a specialized **accessor**, available only on genuine datetime columns, that exposes date-specific properties — `.month` extracts just the month number (1–12) from each full date. Say explicitly: this is analogous to Module 11’s `.get()` or Module 13’s `.str` accessor — a small namespace of extra functionality attached to a column of a specific type.
- `df.groupby("month")["revenue"].sum()` — Exercise 1’s exact aggregation pattern, grouping by month instead of region.

Run this intermediate version and look at the output together — if your dataset happens to have activity in every month, this step won’t yet reveal the gotcha, so narrate it explicitly regardless: “Right now, `monthly` only contains rows for months that actually have at least one order. If, say, February had zero orders in the raw data, February simply wouldn’t appear in this result at all — not as a zero, just *absent*. And the exercise requires all 12 months to show on the x-axis, absent or not.”

Now the fix and the full chart:

```

monthly = monthly.reindex(range(1, 13), fill_value=0)
print(monthly)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(monthly.index, monthly.values, marker='o')
ax.set_xticks(range(1, 13))
ax.set_title("Monthly Revenue Trend")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q2_monthly_trend.png", dpi=150)
plt.show()

```

Line-by-line explanation of the fix:

- `.reindex(range(1, 13), fill_value=0)` — **this is the exercise’s real payoff line.** `.reindex()` forces the result to have exactly the index values listed (1 through 12, i.e. every month), inserting `fill_value=0` for any that were missing from the original grouped result. Say explicitly: **this is the general, reliable pattern for “make sure every expected category appears, even ones with zero activity”** — it applies just as well to regions, products, or any other category that might legitimately have zero rows in a given dataset.
- `ax.set_xticks(range(1, 13))` — explicitly forces the x-axis to show a tick mark for every month number, 1 through 12, regardless of how many distinct values are actually present in the plotted data — a second, chart-level safeguard reinforcing the same “show every expected period” requirement.
- `marker='o'` — required by the exercise: circular markers at each actual data point, distinguishing “we have a real data point here” from just the connecting line between points.

Run it. Expected output (verified against the synthetic dataset — a deliberate December seasonal peak):

month	
1	23623.82
2	29972.71
3	24768.20
4	23804.40
5	31168.51
6	34053.23
7	24464.09
8	37767.93
9	29883.33
10	30802.71
11	40491.39

12 63641.59

Required one-sentence finding: “December was the peak month at \$63,641.59 in revenue, about 94% above the monthly average — consistent with a genuine holiday-season sales pattern.”

Common student mistakes to watch for:

- Skipping `.reindex()` entirely because “it happened to work anyway” on their specific dataset (every month having at least one order by chance) — this is a real risk worth naming explicitly: a script that *coincidentally* produces correct-looking output on one dataset but is missing a genuinely necessary safeguard is a fragile script, likely to break silently on the capstone’s actual dataset or any future month with a genuine sales gap.
- Calling `.dt.month` on a column that’s still object dtype (forgot the `pd.to_datetime()` conversion first) — raises `AttributeError: Can only use .dt accessor with datetimelike values`, a good, specific error worth reading together since it directly names the fix.

Check for understanding: “If March had genuinely zero orders in the real data, what would monthly show for March *before* `.reindex()`, versus *after*?” (Before: March simply wouldn’t be a row in the result at all — not zero, absent. After: March would show explicitly as 0. Getting a student to articulate “absent” as different from “present with value zero” is the exercise’s real conceptual point.)

6.4 Exercise 3 — Q3: Order Size Distribution (0:24–0:33, 9 min)

Teaching goal: A histogram — a genuinely different chart *type* from Exercises 1–2’s bar/line charts, answering a different *kind* of question (not “how much per category,” but “what does the overall shape of this variable look like”) — plus a direct, concrete connection to Module 14’s mean-vs-median skew discussion.

Say to the class:

“Different question entirely: not ‘how much per region,’ but ‘what does the spread of individual order sizes actually look like?’ This calls for a histogram, a genuinely different chart type — and it connects directly to something we noticed while cleaning this data last module.”

Live-code this:

```
# --- Exercise 3 (Q3) ---
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(df['revenue'], bins=20, color="#55A868", edgecolor="white")
ax.set_title("Order Size Distribution")
ax.set_xlabel("Revenue ($)")
ax.set_ylabel("Number of Orders")
plt.tight_layout()
plt.savefig("module15/charts/q3_order_distribution.png", dpi=150)
plt.show()

median = df['revenue'].median()
mean = df['revenue'].mean()
print(f"Median: ${median:.2f}, Mean: ${mean:.2f}")
```

Line-by-line explanation:

- `ax.hist(df['revenue'], bins=20, ...)` — a **histogram**, required by the exercise, not a bar chart — say explicitly why these are genuinely different, not just visually similar: a bar chart’s bars each represent one *category* (a region, a month); a histogram’s bars (“bins”) each represent a *range of numeric values*, and the bar height shows how many individual orders fall into that range. `bins=20` divides the full range of revenue values into 20 equal-width buckets.
- `median = df['revenue'].median(), mean = df['revenue'].mean()` — two different measures of “the center” of the distribution, computed directly for comparison against each other.

Run it. Expected output (verified against the synthetic dataset):

Median: \$194.87, Mean: \$328.70

Walk through the required interpretation, since it’s this exercise’s actual point: “The mean (\$328.70) is noticeably *higher* than the median (\$194.87). What does that tell you about the shape

of this distribution?” — get the room to reason toward: **a mean pulled well above the median indicates a right-skewed distribution** — most orders are relatively small, but a smaller number of large orders pull the average upward. Connect this explicitly back to Module 14 Exercise 6: this is the exact same signal (mean above median) students were asked to notice in `.describe()`’s output last week, now visualized directly as a histogram’s shape — a long tail stretching to the right, with most of the bars clustered on the lower end.

Required one-sentence finding: “Order revenue is right-skewed (mean \$328.70 vs. median \$194.87) — a small number of large orders pull the average well above what a ‘typical’ order actually looks like, which means the median is the more representative single number for describing a typical order here, not the mean.”

Common student mistakes to watch for:

- Building a bar chart instead of a true histogram — e.g., manually binning the data and using `ax.bar(...)` — not conceptually wrong, but the exercise specifically asks for `bins=20` on `ax.hist(...)`, and it’s worth having students use the purpose-built tool rather than reinventing it.
- Stating the skew direction backwards — a common mix-up worth drilling: **right-skewed means the tail (the long stretch of less-common, extreme values) points to the right**, toward higher values, which is also where the mean gets pulled relative to the median. If a student says “left-skewed” when the mean is above the median, walk through the definition again rather than just correcting the word.
- Choosing mean over median as “the” typical order size in their written finding, contradicting their own correct observation about skew — a good moment to explicitly connect the *business* implication (which number should an analyst actually report as “typical”) to the statistical observation, not just note the skew as a fact in isolation.

Check for understanding: “If this distribution were instead perfectly symmetric, what relationship would you expect between the mean and the median?” (They’d be equal, or very close — no skew pulling the average away from the middle value. A good check that the mean-vs-median-as-a-skew-signal idea generalizes beyond this one specific dataset.)

6.5 Exercise 4 — Q4: Your Own Business Question (0:33–0:41, 8 min)

Teaching goal: Independently formulate a genuine business question, choose an appropriate chart type for it, and justify that choice in writing — the exercise most directly rehearsing the capstone’s own open-ended requirements.

Say to the class:

“Now you pick the question. Something a real analyst or manager would actually want to know from this dataset — not just ‘because I can compute it.’ And I want a comment justifying why you chose the chart type you did, not just which one.”

Facilitation notes rather than a single live-coded answer — this is intentionally open-ended, and modeling just one “correct” version risks the room converging on a single unoriginal idea:

- Circulate and ask each student or pair to state their question **out loud, in one sentence**, before they start coding it — this catches a genuinely common failure mode early: a “question” that’s actually just a restatement of a groupby operation (“group by product category”) rather than an actual business question (“which product category should we stock more of heading into the holiday season, based on this year’s trend?”).
- If a student is stuck, offer a few genuine starter angles without fully solving it for them: revenue by product category (bar chart), order count by day-of-week (`.dt.dayofweek`, bar chart — a genuinely interesting extension of Exercise 2’s date-accessor skill), or a scatter of `quantity_sold` vs. `unit_price` (do larger orders tend to be for cheaper or pricier items?).
- The chart-type-justification comment is graded content, per the lab page — model a strong one explicitly: “I chose a bar chart, not a line chart, because product category is a discrete set of unordered labels, not a continuous sequence like time — a line chart would misleadingly imply an ordering or trend between categories that don’t actually have one.”

Example demonstration, framed explicitly as “one possible answer, not the answer” (verified against the synthetic dataset):

```
# --- Exercise 4 (Q4) ---
# Business question: which product category generates the most revenue?
# Chart type: bar chart – product_category is a discrete, unordered set
# of labels, so a bar chart (not a line chart, which would imply an
# ordering/trend that doesn't exist between categories) is the right fit.
cat_revenue = df.groupby("product_category")["revenue"].sum().sort_values(ascending=False)
print(cat_revenue)

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(cat_revenue.index, cat_revenue.values, color="#C44E52")
ax.set_title("Total Revenue by Product Category")
ax.set_xlabel("Product Category")
```

```
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q4_custom.png", dpi=150)
plt.show()
```

Verified output:

product_category	
Apparel	103886.52
Home Goods	101113.58
Toys	98238.57
Electronics	91203.24

Common student mistakes to watch for:

- A question that's too broad to answer with one chart ("how is the business doing overall?") — help narrow it to something a single, specific chart can genuinely answer.
- A technically correct chart with a weak or missing business framing — the lab page explicitly grades "quality of the business question," not just execution; push students to state *why* their chosen manager or analyst would care about the answer, not just that the chart is accurate.

Check for understanding: "If a colleague looked at your Q4 chart for ten seconds with no other context, could they state your finding?" (This is directly lifted from tonight's own first reflection question — asking it now, per-chart, in the moment, is better rehearsal than only reflecting on it once at the end.)

6.6 Exercise 5 — Seaborn Version (0:41–0:48, 7 min)

Teaching goal: Rebuild one existing chart in Seaborn instead of matplotlib, and articulate a genuine, specific difference — not just “it looks nicer.”

Say to the class:

“Seaborn is built on top of matplotlib — think of it as a layer that handles a lot of styling and statistical-chart conveniences automatically. We’re rebuilding Q1’s bar chart as a Seaborn version, and I want a real, specific observation about what’s different, not just a vibes-based ‘it looks better.’”

Live-code this:

```
# --- Exercise 5 ---
import seaborn as sns

fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(data=region_revenue, x="region", y="revenue", ax=ax, color="#4C72B0")
ax.set_title("Total Revenue by Region (Seaborn)")
ax.set_xlabel("Region")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q1_seaborn.png", dpi=150)
plt.show()

# Difference observed: sns.barplot() took a DataFrame and column names
# directly (data=, x=, y=) instead of separate x/y arrays – noticeably
# less code to point at the right values, and the default color palette
# and gridlines needed no manual styling to look presentable.
```

Line-by-line explanation:

- `import seaborn as sns` — same aliasing convention as `pandas as pd`; `sns` is the near-universal community convention for Seaborn.
- `sns.barplot(data=region_revenue, x="region", y="revenue", ax=ax, ...)` — note the calling convention is genuinely different from matplotlib’s `ax.bar(...)`: Seaborn functions typically take the whole DataFrame via `data=`, plus column *names* (as strings) for `x=`/`y=`, rather than requiring you to pull out and pass raw arrays yourself. Say explicitly: this is often less code for the same result, especially as the number of variables involved grows (a Seaborn chart with a third `hue=` grouping variable, for instance, stays almost as simple to write — the equivalent in raw matplotlib gets noticeably more verbose).
- `ax=ax` — Seaborn functions can still be told which matplotlib axes to draw onto, which is how this integrates cleanly with the same `fig, ax = plt.subplots(...)` pattern every other chart in this lab already uses — worth stating explicitly that Seaborn isn’t a wholesale

replacement for matplotlib, it's a complementary layer on top of it.

Run it and compare the two Q1 charts side by side (the original `q1_revenue_by_region.png` and this new `q1_seaborn.png`) — have the room genuinely look at both before writing their comment.

Common student mistakes to watch for:

- A comment that only notes an aesthetic difference (“the seaborn one looks nicer”) without anything more specific — push for *what specifically* looks different (default color choices, background gridlines, bar edge styling) or a code-volume observation (how many lines/arguments were needed for equivalent output).
- Forgetting `ax=ax` and letting Seaborn create its own default figure — not wrong, exactly, but breaks the consistent `fig, ax` pattern the rest of the lab uses, and can cause confusion about which figure ultimately gets saved by `plt.savefig(...)`.

Check for understanding: “If you needed a chart broken down by region *and* product category at once — a grouped or stacked bar chart — which library would likely need less code to get there, based on what you just saw?” (Seaborn — its `hue=` parameter handles this kind of multi-dimensional grouping with one additional argument, where equivalent raw matplotlib code typically requires manually computing bar positions and offsets. Not something to demonstrate live necessarily, but a good forward-looking question.)

6.7 Exercise 6 — Multiple Aggregations (0:48–0:55, 7 min)

Teaching goal: `.agg()` to compute more than one statistic from a single `groupby` call — and a genuine business insight that “most orders” and “highest average order value” can point to *different* regions, a distinction worth computing explicitly rather than assuming.

Say to the class:

“One groupby, two statistics at once — total revenue and order count, per region. And I want you to genuinely check: does the region with the most orders also have the highest average order value, or are those two different regions? Don’t assume — compute it.”

Live-code this:

```
# --- Exercise 6 ---
region_stats = df.groupby("region").agg(
    total_revenue=("revenue", "sum"),
    order_count=("revenue", "count"),
)
region_stats["avg_order_value"] = region_stats["total_revenue"] / region_stats["order_count"]
print(region_stats)

# Most orders: [fill in after reading output]
# Highest average order value: [fill in after reading output]
```

Line-by-line explanation:

- `df.groupby("region").agg(total_revenue=("revenue", "sum"), order_count=("revenue", "count"))` — **named aggregation syntax**, worth reading carefully: each keyword argument (`total_revenue=`, `order_count=`) names a new **output column**, and its value is a tuple of (source_column, aggregation_function). Say explicitly: this single call is doing what would otherwise require two separate `groupby` calls (one `.sum()`, one `.count()`) merged back together — `.agg()` computes both from the same grouping in one pass.
- `("revenue", "count")` — note this counts how many revenue values exist per group, which — since every row has a revenue value — is equivalent to counting *rows* per region, i.e., the number of orders.
- `region_stats["avg_order_value"] = region_stats["total_revenue"] / region_stats["order_count"]` — Module 13’s vectorized new-column pattern, now computing a *derived* statistic from two columns that were themselves just computed by the aggregation above.

Run it. Expected output (verified against the synthetic dataset — deliberately designed so these two questions have different answers):

```
      total_revenue  order_count  avg_order_value
region
```


East	158621.95	311	510.038424
North	60112.83	241	249.430830
South	112040.69	403	278.016600
West	63666.44	245	259.863020

Walk through the required comparison explicitly, since it's the exercise's real point: South has the **most orders** (403), but **East** has the **highest average order value** (\$510.04) — and, from Exercise 1, East also has the highest *total* revenue, despite having *fewer* orders than South. Say plainly: **this is a genuinely useful, non-obvious business insight** — East isn't winning on volume, it's winning because each individual order tends to be much larger. A manager deciding where to invest marketing spend to *grow order count* versus where to invest in *increasing average order size* would want to know these are two different regions with two different opportunities, not the same region needing the same intervention.

Common student mistakes to watch for:

- Assuming “most orders” and “highest average value” must be the same region without actually checking both numbers — this is exactly the assumption the exercise is designed to test; if a student's answer just states one region for both without computing `avg_order_value` explicitly, that's a good moment to have them verify rather than assume.
- Computing `avg_order_value` incorrectly as `total_revenue.mean()` or similar, rather than a genuine row-by-row division of the two aggregated columns — worth a quick sanity check: does the computed average order value roughly match what dividing the printed `total_revenue` by `order_count` gives by hand for at least one region?

Check for understanding: “Why might a region have a high average order value but relatively few total orders — what's a plausible business explanation?” (A few genuinely plausible answers: fewer, larger institutional/bulk customers rather than many small individual ones; a regional focus on higher-priced product categories; less overall market penetration but stronger performance where it does sell. No single right answer — the point is generating a plausible, business-grounded hypothesis, not just noticing the number.)

6.8 Exercise 7 — Notebook Tidy-Up (0:55–0:59, 4 min)

Teaching goal: A quick, low-cognitive-load organizational pass — matching Module 15's “capstone dress rehearsal” framing, since the capstone itself will expect a clearly organized, navigable submission, not just correct code buried in one long undifferentiated block.

Say to the class:

“Quick housekeeping pass, not new material: organize what you've already built so someone opening this file cold can navigate it.”

If working in Jupyter: add a markdown cell before each chart block with a level-2 heading and a

one-sentence business question, e.g.:

```
## Q1: Which region generates the most revenue?
```

If working in a .py file: add an equivalent section comment block before each chart, matching the pattern the guide's own answer key uses throughout (# --- Exercise N (QN) ---), extended with a one-line business-question restatement.

Common student mistakes to watch for:

- Treating this as purely cosmetic and rushing it — worth stating explicitly that a well-organized submission is itself part of the capstone's actual grading standard, not a nice-to-have; five minutes now is genuinely worth it.

Check for understanding: No dedicated check — this exercise is naturally verified simply by looking at the final file/notebook's readability together as a class, or spot-checking a couple of students' screens as you circulate.

7 Wrap-Up (last ~6 minutes)

Review the reflection questions out loud:

1. *Could someone unfamiliar with the dataset state your key finding from a 10-second look at each chart?* — a strong, honest self-assessment names a specific chart that would benefit from a clearer title or a more prominent label, not just “yes, all of them” reflexively.
2. *Practice writing the “\$Y, Z% above average” sentence for each required chart* — this has already been rehearsed live for Exercises 1–3 in this session; have students write all three versions down now, fresh, without looking back at the board.
3. *What felt difficult or unclear — note it now* — genuinely useful information both for the student (heading into the capstone) and for you as the instructor (a good signal for what to reinforce in Module 16’s dedicated capstone work time).

Review the submission checklist together:

- ☐ `aggregate.ipynb` (or `.py`) contains all 7 exercises, clearly organized (Exercise 7’s tidy-up)
- ☐ All 4 required PNG charts saved in `module15/charts/`: `q1_revenue_by_region.png`, `q2_monthly_trend.png`, `q3_order_distribution.png`, `q4_custom.png`
- ☐ Every required chart has a title, axis labels, and a one-sentence numeric finding
- ☐ Q2’s chart shows all 12 months, even any with zero revenue
- ☐ Pushed to GitHub inside a `module15/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 16: “This lab was the dress rehearsal — same workflow, same standards, smaller scale. Module 16 is dedicated capstone work time: a longer version of exactly what you just did, on a dataset and set of business questions of your own. Everything that felt unclear today is worth resolving before then, not during.”

8 Appendix A — Full Answer Key (`aggregate.py`)

```
# aggregate.py
# ISM2411 Module 15 Lab – Aggregate & Chart: Capstone Warm-up
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('data/clean_sales.csv')
df['order_date'] = pd.to_datetime(df['order_date'])

# --- Exercise 1 (Q1): Revenue by region ---
region_revenue = df.groupby("region")["revenue"].sum().reset_index()
```

```

region_revenue = region_revenue.sort_values("revenue", ascending=False)
print(region_revenue)

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(region_revenue["region"], region_revenue["revenue"], color="#4C72B0")
ax.set_title("Total Revenue by Region")
ax.set_xlabel("Region")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q1_revenue_by_region.png", dpi=150)
# Finding: East drove $158,621.95 in revenue, the most of any region –
# about 41% more than second-place South.

# --- Exercise 2 (Q2): Monthly revenue trend ---
df['month'] = df['order_date'].dt.month
monthly = df.groupby("month")["revenue"].sum().reindex(range(1, 13), fill_value=0)
print(monthly)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(monthly.index, monthly.values, marker='o')
ax.set_xticks(range(1, 13))
ax.set_title("Monthly Revenue Trend")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q2_monthly_trend.png", dpi=150)
# Finding: December is the peak month at $63,641.59, about 94% above
# the monthly average – a genuine holiday-season seasonal pattern.

# --- Exercise 3 (Q3): Order size distribution ---
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(df['revenue'], bins=20, color="#55A868", edgecolor="white")
ax.set_title("Order Size Distribution")
ax.set_xlabel("Revenue ($)")
ax.set_ylabel("Number of Orders")
plt.tight_layout()
plt.savefig("module15/charts/q3_order_distribution.png", dpi=150)

median = df['revenue'].median()
mean = df['revenue'].mean()
print(f"Median: ${median:.2f}, Mean: ${mean:.2f}")

```

```

# Finding: right-skewed (mean $328.70 vs median $194.87) – a few large
# orders pull the average up; the median better represents a typical order.

# --- Exercise 4 (Q4): Custom business question ---
# Business question: which product category generates the most revenue?
# Chart type: bar chart – product_category is discrete/unordered, so a
# bar chart (not a line chart, which would imply an ordering) fits.
cat_revenue = df.groupby("product_category")["revenue"].sum().sort_values(ascending=False)
print(cat_revenue)

fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(cat_revenue.index, cat_revenue.values, color="#C44E52")
ax.set_title("Total Revenue by Product Category")
ax.set_xlabel("Product Category")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q4_custom.png", dpi=150)

# --- Exercise 5: Seaborn version of Q1 ---
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(data=region_revenue, x="region", y="revenue", ax=ax, color="#4C72B0")
ax.set_title("Total Revenue by Region (Seaborn)")
ax.set_xlabel("Region")
ax.set_ylabel("Revenue ($)")
plt.tight_layout()
plt.savefig("module15/charts/q1_seaborn.png", dpi=150)
# Difference: sns.barplot() takes data=/x=/y= column names directly
# instead of raw arrays; noticeably less code, sensible default styling.

# --- Exercise 6: Multiple aggregations ---
region_stats = df.groupby("region").agg(
    total_revenue=("revenue", "sum"),
    order_count=("revenue", "count"),
)
region_stats["avg_order_value"] = region_stats["total_revenue"] / region_stats["order_count"]
print(region_stats)
# Most orders: South (403). Highest average order value: East ($510.04).

```

9 Appendix B — Reproducible Full-Year Dataset (for instructor testing)

A generator producing a 1,200-row, full-calendar-year dataset with a genuine December seasonal peak and a deliberate region split — South has the most orders, but East wins on both total revenue and average order value — specifically so Exercise 6's two questions have two different, non-obvious answers. **Use your course's real Module 14 output with students.**

```
import pandas as pd
import numpy as np

rng = np.random.default_rng(99)
n = 1200
regions = ["North", "South", "East", "West"]
categories = ["Electronics", "Home Goods", "Apparel", "Toys"]

region_arr = rng.choice(regions, size=n, p=[0.22, 0.34, 0.24, 0.20])
category_arr = rng.choice(categories, size=n)
quantity_arr = rng.integers(1, 15, size=n)

base_prices = np.array([9.99, 14.99, 19.99, 24.99, 49.99, 99.99])
unit_price = np.zeros(n)
for i in range(n):
    if region_arr[i] == "East":
        unit_price[i] = rng.choice([19.99, 24.99, 49.99, 99.99, 149.99])
    else:
        unit_price[i] = rng.choice(base_prices)

dates = pd.date_range("2024-01-01", "2024-12-31", freq="D")
weights = np.ones(len(dates))
for i, d in enumerate(dates):
    if d.month == 12:
        weights[i] = 2.2
    elif d.month == 11:
        weights[i] = 1.4
weights = weights / weights.sum()
date_arr = rng.choice(dates, size=n, p=weights)

revenue_arr = np.round(quantity_arr * unit_price, 2)
order_id = [f"ORD{100001+i}" for i in range(n)]

df = pd.DataFrame({
    "order_id": order_id,
    "order_date": pd.to_datetime(date_arr).strftime("%Y-%m-%d"),
    "region": region_arr,
```

```
"product_category": category_arr,  
"unit_price": unit_price,  
"quantity_sold": quantity_arr,  
"revenue": revenue_arr,  
})  
df.to_csv("data/clean_sales.csv", index=False)
```

All expected output, chart data, and findings shown throughout this guide were computed and verified against this exact generator's output.

10 Appendix C — Extra Practice (only if the class finishes early)

Seven required exercises fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a second single-dimension aggregation, different grouping. Have students compute and chart revenue by `product_category` broken down further with `.agg()` to also show order count per category, then answer: does the category with the most orders also have the highest total revenue? (Verified: highest revenue is Apparel at \$103,886.52; a good independent check of whether “most popular” and “most revenue” align for categories the same way Exercise 6 explored for regions.)

Extra — a day-of-week chart. Using `df['order_date'].dt.day_name()`, have students group revenue by day of week and build a bar chart — a good extra rep of the `.dt` accessor from Exercise 2, applied to a genuinely different, real business question (are weekend orders bigger or smaller than weekday orders?).